

CodePipeline and CI/CD

CodePipeline · CodeBuild · CodeDeploy · Blue/Green · Docker · ShopFlow

AWS Tutorial · ShopFlow

16-01

Why CI/CD Matters for ShopFlow

■ CI/CD: Continuous Integration and Continuous Delivery

CI/CD is the practice of automatically building, testing, and deploying code every time a developer pushes a change. "CI" (Continuous Integration) means merging code frequently and running automated tests immediately. "CD" (Continuous Delivery) means automatically deploying passing builds to production without manual steps. The goal: go from code commit to production in minutes, not weeks.

Example: ShopFlow deploys 15-20 times per day. Each engineer pushes a feature branch, the pipeline runs 400+ tests in 8 minutes, a reviewer approves the PR, and merge to main triggers automatic deployment to staging and then production — all with zero manual commands. Before CI/CD, ShopFlow did monthly releases with hours of manual testing and frequent rollbacks.

Without CI/CD	With CI/CD (ShopFlow)
Manual builds: developer runs mvn package on laptop	Automated builds: every push triggers CodeBuild in 8 minutes
Monthly releases with big-bang deployments (high risk)	15-20 deployments per day (small, low-risk changes)
"Works on my machine" — environment differences cause bugs	Consistent build environment (Docker) in CodeBuild — same on every machine
Manual testing before each release (2-3 days)	400+ automated tests run on every commit in 8 minutes
Rollback by reverting a 2-week sprint (painful)	Rollback by reverting one small commit (ECS deployment takes 3 min)
Fear of deploying on Friday afternoon	Deploy any time — pipeline and tests catch problems before production

16-02

AWS CI/CD Services Overview

AWS provides a suite of CI/CD services. ShopFlow uses all of them together as one pipeline.

Service	Role in CI/CD	ShopFlow Use
CodeCommit	Git repository hosted in AWS (similar to GitHub)	ShopFlow uses GitHub (not CodeCommit) — but CodePipeline supports CodeCommit
CodeBuild	Managed build service: run mvn test, docker build, Shell scripts, Maven shell tests, integrations tests (Docker version manual)	ShopFlow: Maven shell tests, integrations tests (Docker version manual)
CodeDeploy	Managed deployment service: rolling, blue/green, Simple file deployments, ECS deployments	ShopFlow: blue/green ECS deployments (10% at a time)
CodePipeline	Orchestrator: connects source (GitHub) -> CodeBuild -> CodeDeploy	ShopFlow: Deploy pipeline as a pipeline -> Build stage -> Test stage -> Deploy stage
Amazon ECR	Docker image registry: store and version Docker images	ShopFlow: ECS product-svc, order-svc, cart-svc Docker images stored in ECR

16-03

ShopFlow CI/CD Pipeline — 5 Stages

Here is ShopFlow's complete pipeline flow. Every push to the "main" branch in GitHub triggers this pipeline automatically.

Stage	What Happens	Duration	Pass/Fail Condition
1. Source	CodePipeline detects push to GitHub main branch, downloads source code	1 min	Always passes (source available)
2. Build	CodeBuild: mvn package (compile + unit tests) + Docker build + docker push to ECR	46 min	Fail if any of the tests fail or Docker build fails; pipeline continues if successful

Stage	What Happens	Duration	Pass/Fail Condition
3. Integration Test	CodeBuild: start test ECS environment; run integration tests against AWS services (S3, IAM, DB, Aurora); code deploy	10-12 minutes	Fail if AWS services fail or tests fail
4. Staging Deploy	CodeDeploy: blue/green deployment to staging ECS cluster; run smoke tests against staging URL	5-8 minutes	Fail if smoke tests fail or ECS health check fails
5. Prod Deploy	Manual approval gate (SRE lead clicks Approve); CodeDeploy: approve to prod; 10-minute observation	10 minutes	Fail if prod deployment fails or observation fails

INFO Pipeline vs Deployment Frequency

This 5-stage pipeline takes 25-40 minutes end-to-end. ShopFlow engineers do not wait — they push code and continue other work. When the pipeline succeeds, Slack gets a notification. When it fails, the engineer who pushed the breaking change is notified with a link to the failed CodeBuild log. The pipeline is non-blocking for the team but ensures nothing broken reaches production.

16-04

CodeBuild buildspec.yml — The Build Script

The buildspec.yml file tells CodeBuild what to do at each phase. ShopFlow has one buildspec per microservice (product-svc has its own, order-svc has its own) stored at the root of each service's directory.

```
# buildspec.yml - ShopFlow product-svc build definition version: 0.2 # Environment variables
(set in CodeBuild project config or CodePipeline action) env: variables: SERVICE_NAME:
product-svc AWS_REGION: us-east-1 parameter-store: # fetch secrets from SSM Parameter Store
ECR_REPO_URI: /shopflow/ecr/product-svc/uri exported-variables: # pass to later pipeline stages
- IMAGE_TAG phases: install: runtime-versions: java: corretto21 # Java 21 (Amazon Corretto)
commands: - echo "Installing dependencies..." - mvn dependency:go-offline -q # cache Maven deps
for speed pre_build: commands: - echo "Running unit tests..." - mvn test -q # fail here if tests
fail - echo "Logging into ECR..." - aws ecr get-login-password --region $AWS_REGION | docker
login --username AWS --password-stdin $ECR_REPO_URI - IMAGE_TAG=$(echo
$CODEBUILD_RESOLVED_SOURCE_VERSION | cut -c1-7) # git SHA prefix - echo "Building image tag
$IMAGE_TAG" build: commands: - echo "Building JAR..." - mvn package -DskipTests -q # JAR already
built; skip tests (ran in pre_build) - echo "Building Docker image..." - docker build -t
$ECR_REPO_URI:$IMAGE_TAG -t $ECR_REPO_URI:latest . - echo "Pushing Docker image to ECR..." -
docker push $ECR_REPO_URI:$IMAGE_TAG - docker push $ECR_REPO_URI:latest post_build: commands: -
echo "Generating ECS imagedefinitions.json..." - printf '["name":"app","imageUri":"%s"]'
$ECR_REPO_URI:$IMAGE_TAG > imagedefinitions.json - echo "Build completed at $(date)" artifacts:
# output artifacts passed to next pipeline stage files: - imagedefinitions.json # tells
CodeDeploy which Docker image to deploy - taskdef.json # ECS task definition template -
appspec.yaml # CodeDeploy blue/green instructions
```

16-05

CodePipeline Console — Creating a Pipeline

Follow these steps to create the ShopFlow product-svc pipeline in the AWS Console. This is a one-time setup; after that, the pipeline runs automatically on every git push.

■ Create ShopFlow Product-Svc Pipeline

1. AWS Console -> CodePipeline -> Pipelines -> Create pipeline
2. Pipeline name: shopflow-product-svc-pipeline | Execution mode: Superseded (newest push wins)
3. Service role: New service role (auto-created) -> Next
4. Source stage: Provider = GitHub (Version 2) -> Connect to GitHub -> Authorize
5. Repository: shopflow-org/shopflow-backend | Branch: main | Output: SourceArtifact -> Next
6. Build stage: Provider = AWS CodeBuild -> Create project (opens CodeBuild console in new tab)
7. CodeBuild: project name = shopflow-product-svc-build | Runtime = Standard | Image = aws/codebuild/standard:7.0
8. Environment: Managed image | OS = Amazon Linux 2023 | Privileged = YES (needed for docker build)
9. Buildspec: Use a buildspec file (buildspec.yml in repo root)
10. Back in CodePipeline: select shopflow-product-svc-build -> Next
11. Deploy stage: Provider = Amazon ECS (Blue/Green) -> Application: shopflow-prod | Group: product-svc
12. Input: BuildArtifact | Task definition file: taskdef.json | Appspec: appspec.yaml -> Next -> Create pipeline

■ aws > CodePipeline > shopflow-product-svc-pipeline

Pipeline: shopflow-product-svc-pipeline

■ Pipeline Status — Last Run: 14:23 UTC (triggered by GitHub push)

- Stage 1: Source | GitHub main | 0m 28s | SUCCEEDED
- Stage 2: Build | CodeBuild | 5m 12s | SUCCEEDED
- Stage 3: IntegrationTest | CodeBuild | 9m 44s | SUCCEEDED
- Stage 4: StagingDeploy | ECS Blue/Green | 6m 01s | SUCCEEDED
- Stage 5: ManualApproval | Waiting for approval | Notified: sre-lead@shopflow.com

■ Stage 5: ProdDeploy (pending approval)

Approval comment (optional)

Verified staging — checkout flow works correctly

Approve

16-06

Blue/Green ECS Deployment — Zero-Downtime Releases

Blue/Green deployment runs the new version (green) alongside the old version (blue) before switching traffic. If green has problems during the observation window, traffic automatically reverts to blue. ShopFlow uses this for every production deployment.

Phase	What Happens	Duration
Pre-traffic	CodeDeploy creates new ECS task set (green) with new Docker image. Green ECS task set passes health check.	24.6m (50s)
Traffic shift start	CodeDeploy shifts a percentage of ALB traffic from blue to green. ShopFlow uses "All At Once" strategy.	10m (10s)
Observation window	Both blue and green task sets exist. CloudWatch monitors: ALB 5xx, ECS health check failures. If threshold is exceeded, traffic reverts to blue.	10m (10s)

Phase	What Happens	Duration
Traffic finalization	Observation window passes without alarms. CodeDeploy terminates instances (blue tasks) so only the new green tasks remain.	300 seconds
Connection draining	Blue ECS tasks continue to serve in-flight requests for 300 seconds (connection draining) before being terminated.	300 seconds

16-07 CodeDeploy AppSpec and Task Definition

Two files control the CodeDeploy blue/green ECS deployment: `appspec.yaml` (deployment configuration) and `taskdef.json` (ECS task definition template with image placeholder).

```
# appspec.yaml – CodeDeploy blue/green ECS configuration version: 0.0
Resources: -
TargetService: Type: AWS::ECS::Service Properties: TaskDefinition: # placeholder; replaced by
CodeDeploy LoadBalancerInfo: ContainerName: app ContainerPort: 8080 PlatformVersion: LATEST
Hooks: # optional Lambda hooks at each deployment phase - AfterAllowTestTraffic: # runs after
test traffic (before production traffic) - Location: scripts/validate-service.sh # or a Lambda
ARN Timeout: 60 Runas: root

// taskdef.json – ECS task definition template (image URI is placeholder) { "family":
"shopflow-product-svc", "networkMode": "awsvpc", "requiresCompatibilities": ["FARGATE"], "cpu":
"512", "memory": "1024", "executionRoleArn":
"arn:aws:iam::123456789:role/shopflow-ecs-execution-role", "taskRoleArn":
"arn:aws:iam::123456789:role/shopflow-product-svc-task-role", "containerDefinitions": [{
"name": "app", "image": "", // placeholder – CodeDeploy fills this from imagedefinitions.json
"portMappings": [{"containerPort": 8080}], "logConfiguration": { "logDriver": "awslogs",
"options": { "awslogs-group": "/ecs/shopflow/product-svc", "awslogs-region": "us-east-1",
"awslogs-stream-prefix": "product-svc" } }, "secrets": [ { "name": "SPRING_DATASOURCE_PASSWORD",
"valueFrom": "/shopflow/prod/aurora/password" } ] } ] }
```

16-08 CDK Pipeline Code — ShopFlow Full Pipeline

ShopFlow defines the entire CodePipeline in CDK code (PipelineStack). This is a "self-mutating" pipeline — when you change the pipeline CDK code and push, the pipeline updates itself.

```
// PipelineStack.java: ShopFlow CodePipeline in CDK
public class PipelineStack extends Stack {
    public PipelineStack(App app, String id, StackProps props) { super(app, id, props); // Source:
    GitHub Artifact source = new Artifact("Source"); GitHubSourceAction githubSource =
    GitHubSourceAction.Builder.create().actionName("GitHub_Source").owner("shopflow-org")
    .repo("shopflow-backend").branch("main")
    .oAuthToken(SecretValue.secretsManager("github-oauth-token")).output(source).build(); //
    Build: CodeBuild Artifact buildOutput = new Artifact("Build"); PipelineProject buildProject =
    PipelineProject.Builder.create(this, "Build").projectName("shopflow-product-svc-build")
    .buildSpec(BuildSpec.fromSourceFilename("buildspec.yml"))
    .environment(BuildEnvironment.builder().buildImage(LinuxBuildImage.STANDARD_7_0)
    .privileged(true) // required for docker build .build().build(); CodeBuildAction buildAction
    = CodeBuildAction.Builder.create().actionName("CodeBuild").project(buildProject)
    .input(source).outputs(List.of(buildOutput)).build(); // Manual approval before prod
    ManualApprovalAction approveAction = ManualApprovalAction.Builder.create()
    .actionName("ApproveProd").notificationTopic(oncallTopic) // sends SNS email to SRE lead
    .additionalInformation("Check staging: https://staging.shopflow.com/health").build(); //
    Pipeline: wire stages together Pipeline.Builder.create(this, "Pipeline")
    .pipelineName("shopflow-product-svc-pipeline").stages(List.of(
    StageProps.builder().stageName("Source").actions(List.of(githubSource)).build(),
    StageProps.builder().stageName("Build").actions(List.of(buildAction)).build(),
    StageProps.builder().stageName("ApproveProduction").actions(List.of(approveAction)).build(),
    StageProps.builder().stageName("DeployProduction").actions(List.of(ecsDeployAction)).build()))
    .build(); }
```

16-09 ShopFlow CI/CD Architecture

Complete view of the ShopFlow CI/CD pipeline and how the components connect.

ShopFlow CI/CD Pipeline

Source Stage

- GitHub — shopflow-org/shopflow-backend (main branch)
- Webhook triggers CodePipeline on every push to main

Build Stage — CodeBuild (Amazon Linux 2023, Java 21)

- mvn test — 400+ unit tests (JUnit 5 + Mockito)
- mvn package — fat JAR with Spring Boot
- docker build — multi-stage Dockerfile (JRE 21 slim)
- docker push — ECR: product-svc:abc1234 + :latest
- Output artifact: imagedefinitions.json + taskdef.json

Integration Test Stage — CodeBuild

- Start test ECS environment (cdk deploy --context env=test)
- Run integration tests: REST Assured + Testcontainers
- Destroy test environment after tests complete

Staging Deploy Stage — CodeDeploy ECS Blue/Green

- Deploy new image to staging ECS cluster
- Run smoke tests: curl https://staging.shopflow.com/health
- 10-minute observation window with CloudWatch monitoring

Manual Approval Gate — SNS email to SRE lead

- SRE verifies staging; clicks Approve in CodePipeline console

Production Deploy Stage — CodeDeploy ECS Blue/Green

- Deploy to prod ECS cluster (shopflow-prod)
- 10-minute observation window; auto-rollback if 5xx > 1%
- Slack notification: deploy succeeded or rolled back

16-10

Dockerfile — Multi-Stage Build for Java 21

ShopFlow uses a multi-stage Docker build: the first stage uses the full JDK to build the JAR, the second stage uses only the slim JRE to run it. This reduces the final image size from ~600 MB to ~180 MB.

```
# Dockerfile — ShopFlow product-svc multi-stage build # Stage 1: Build (uses full JDK — not
included in final image) FROM amazoncorretto:21-alpine AS builder WORKDIR /app COPY pom.xml .
COPY src ./src # Build the Spring Boot fat JAR RUN mvn package -DskipTests -q # Extract Spring
Boot layers for better Docker layer caching RUN java -Djarmode=layertools -jar
target/product-svc.jar extract --destination extracted # Stage 2: Runtime (uses slim JRE —
```

```
final image deployed to ECS) FROM amazoncorretto:21-alpine-jre WORKDIR /app # Copy Spring Boot layers (dependencies rarely change -> cached layer) COPY --from=builder /app/extracted/dependencies/ ./ COPY --from=builder /app/extracted/spring-boot-loader/ ./ COPY --from=builder /app/extracted/snapshot-dependencies/ ./ # Application code (changes most often -> last layer; only this re-downloads on deploy) COPY --from=builder /app/extracted/application/ ./ # Non-root user for security RUN addgroup -S shopflow && adduser -S shopflow -G shopflow USER shopflow # Health check – used by ECS and Docker HEALTHCHECK --interval=30s --timeout=5s --start-period=60s --retries=3 \ CMD wget --quiet --tries=1 http://localhost:8080/actuator/health || exit 1 EXPOSE 8080 # Spring Boot layered JAR entry point ENTRYPOINT ["java", "-XX:+UseContainerSupport", "-XX:MaxRAMPercentage=75.0", "org.springframework.boot.loader.launch.JarLauncher"]
```

16-11

Rollback Strategies — When Deployments Go Wrong

ShopFlow has three rollback mechanisms, used in order of how quickly the problem is detected.

Mechanism	When It Triggers	How It Works	Time to Rollback
CodeDeploy Auto-Rollback	During 10-minute observation window: ALB error rate spikes	CodeDeploy automatically shifts traffic back to blue instances (if available)	2 minutes (if set up correctly)
CodePipeline Pipeline Stop	Before production: staging smoke tests fail	Pipeline stops; production never receives the bad code and deployment is retried	No rollback needed (bad code not deployed)
Manual Git Revert	After observation window: a bug is found	Engineer triggers a new release (e.g. git push) to prod	25-40 minutes (full pipeline run)

```
// CDK: Configure CodeDeploy blue/green with auto-rollback alarms EcsDeploymentGroup deploymentGroup = EcsDeploymentGroup.Builder.create(this, "ProdDG") .service(productFargateService) .blueGreenDeploymentConfig(EcsBlueGreenDeploymentConfig.builder() .blueTargetGroup(blueTargetGroup) .greenTargetGroup(greenTargetGroup) .listener(httpsListener) .deploymentApprovalWaitTime(Duration.minutes(0)) // immediate traffic shift .terminationWaitTime(Duration.minutes(10)) // observation window .build()) .alarms(List.of( // Auto-rollback if 5xx error rate spikes during observation window Alarm.fromAlarmArn(this, "5xxAlarm", albHighErrorAlarm.getAlarmArn()))) .autoRollback(AutoRollbackConfig.builder() .failedDeployment(true) // rollback on deployment failure .deploymentInAlarm(true) // rollback if any alarm fires during observation .build()) .build();
```

16-12

Hands-On Lab — Build and Deploy a Spring Boot Service

In this lab you will create a CodeBuild project, run a build of a simple Spring Boot app, and view the output in CodePipeline. Time estimate: 45 minutes.

■ Lab: CodeBuild + CodePipeline for Spring Boot

1. Create a Spring Boot app: <https://start.spring.io> -> Java 21, Maven, Spring Web, Spring Actuator
2. Add a simple endpoint: GET /hello returns {"message":"Hello from ShopFlow"}
3. Add buildspec.yml at project root with phases: install (java: corretto21), build (mvn package)
4. Push to a new GitHub repository
5. AWS Console -> CodeBuild -> Create build project
6. Source: GitHub -> Connect -> select your repository
7. Environment: Managed image | Amazon Linux 2023 | Standard 7.0 | Privileged = NO (no Docker needed)
8. Buildspec: Use a buildspec file
9. Start build -> watch the build phases in real-time (click "Tail logs")
10. Phase log shows: install -> pre_build -> build -> post_build
11. Build succeeds: green checkmarks for each phase
12. Now create a CodePipeline: Source = GitHub, Build = your CodeBuild project
13. Push a code change -> watch pipeline trigger automatically

16-13

Branch Strategy — Feature Flags and Trunk-Based Development

ShopFlow uses trunk-based development: engineers commit to short-lived feature branches and merge to "main" multiple times per day. Long-lived branches cause merge conflicts and deployment complexity.

Branch Strategy	Description	ShopFlow Decision
Gitflow (feature/develop/releases/main)	feature branches; release branches; complex merge process	Not used: complex; long feature branches cause painful merges
GitHub Flow (feature/main)	short feature branches; PR to main; deploy on merge	Not used directly: no staging/prod environment differentiation
Trunk-Based Development (ShopFlow)	All (ShopFlow) short branches (< 2 days); merge to trunk multiple times per day	Used: simple/fast deployment; feature flags control feature visibility

Feature flags let ShopFlow deploy code to production without making a feature visible to users. The new checkout flow is deployed in 3 phases:

```
// ShopFlow: Feature flag for new checkout flow // Spring Boot service reads flag from AWS
AppConfig (or simple SSM Parameter) @Service public class CheckoutService { @Autowired private
FeatureFlagService flags; public CheckoutResponse checkout(CartId cartId, UserId userId) { if
(flags.isEnabled("new-checkout-flow", userId)) { // New checkout: React + Stripe Elements (50%
of users) return newCheckoutService.checkout(cartId, userId); } else { // Old checkout: legacy
JSP form (50% of users) return legacyCheckoutService.checkout(cartId, userId); } } } // Feature
flag service reads from SSM Parameter Store // /shopflow/flags/new-checkout-flow = {"enabled":
true, "percentage": 50} // Deploy to 10% -> monitor -> 25% -> 50% -> 100% over 2 weeks // If
problems: set percentage to 0 (instant rollback, no code deploy needed)
```

16-14

CodeBuild Advanced — Caching and Parallelism

By default, CodeBuild downloads all Maven dependencies on every build — slow and wasteful. Caching and parallel test execution cut ShopFlow's build time from 18 minutes to 6 minutes.

```
# buildspec.yml additions for caching and parallel tests # Cache Maven repository between builds
(S3 cache) cache: paths: - /root/.m2/**/* # Maven local repository - /root/.docker/**/* # Docker
layer cache # Parallel build matrix: run unit tests across 4 CodeBuild instances simultaneously
# (uses CodeBuild batch builds) batch: fast-fail: true # if any batch instance fails, cancel all
others build-list: - identifier: test_unit env: variables: TEST_SUITE: unit buildspec:
buildspec-unit.yml - identifier: test_integration env: variables: TEST_SUITE: integration
```


buildspec: buildspec-integration.yml - identifier: build_docker buildspec: buildspec-docker.yml - identifier: cdk_synth buildspec: buildspec-cdk.yml # Result: all 4 run in parallel (6 min) instead of sequentially (24 min) # Batch builds cost: 4 build minutes billed as 4 separate CodeBuild minutes

Optimization	Before	After	Savings
Maven cache (S3)	mvn dependency download: 4 min per build	Cached: 20 seconds	3.5 min per build
Docker layer cache	Docker pulls base image each build	Cached layers from S3: 15s	1.5 min per build
Parallel test suites	400 tests sequential: 12 min	4 parallel suites: 3 min each	9 min total
TOTAL build time	18 minutes	6 minutes	67% faster

16-15 Security in CI/CD — SAST, DAST, and Supply Chain

ShopFlow integrates security scanning directly into the pipeline. Every build automatically checks for vulnerabilities in dependencies, secrets accidentally committed, and Docker image CVEs.

Security Check	Tool	When in Pipeline	What It Finds
SAST (Static Analysis)	SpotBugs + FindSecBugs (Maven)	Build stage (mvn verify)	SQL injection, XSS, insecure crypto patterns in Java code
Dependency CVE scan	OWASP Dependency Check (Maven)	Build stage (mvn verify)	Known CVEs in Spring Boot, Jackson, Log4j, and others
Secrets detection	GitLeaks (pre-commit hook + CodeBuild)	Before build	AWS keys, passwords, API tokens accidentally committed
Docker image scan	Amazon ECR image scanning (At Docker push)	After docker push	OS-level CVEs in the JRE base image (e.g., OpenSSH)
DAST (Dynamic Analysis)	OWASP ZAP (lightweight scanner)	Staging deployment	SQL injection, XSS, SSRF against the running staging app

buildspec.yml: add security checks to build phase phases: build: commands: - echo "Running security scans..." # SAST + dependency CVE check (fails build if HIGH severity found) - mvn verify -P security-checks -DfailBuildOnCVSS=7 -q # Secrets detection (fails build if any secrets found in git history) - gitleaks detect --source . --config gitleaks.toml --exit-code 1 # Docker build + push - docker build -t \$ECR_REPO_URI:\$IMAGE_TAG . - docker push \$ECR_REPO_URI:\$IMAGE_TAG # ECR scan results available in console after push # CRITICAL CVEs in ECR scan trigger a separate CloudWatch alarm

16-16 CI/CD Cost Analysis

AWS CI/CD services are priced per use. With 15-20 deploys per day, ShopFlow's pipeline costs are modest compared to the value of fast, reliable deployments.

Service	Pricing	ShopFlow Monthly Usage	Monthly Cost
CodePipeline	\$1.00 per active pipeline/month	4 pipelines (product, order, cart, user)	\$4.00
CodeBuild	\$0.005/build-minute (general1.small); free for 15 deploys/day/month	15 deploys/day x 6 min x 30 days = 2700 build minutes (x4 parallel = 10800)	\$54.00
CodeDeploy	Free for ECS blue/green deployments	N/A	\$0.00
ECR — Storage	\$0.10/GB/month	10 images x 180 MB each = 1.8 GB (per service)	\$0.18 x 4 services = 7.2 GB
ECR — Data transfer	\$0.09/GB (to internet)	ECS pulls from ECR within AWS (same region)	\$0.00 — free
S3 — Build cache	\$0.023/GB/month	2 GB Maven cache per build project x 4	\$0.18
TOTAL	—	15-20 deploys/day x 4 services	~\$59/month

INFO Cost vs Value

ShopFlow deploys 15-20 times per day vs. the industry average of once per week. At \$59/month, that is about \$0.20 per deployment for infrastructure, not counting engineer time. The faster feedback loop (minutes, not weeks) means bugs are caught and fixed same-day instead of accumulating across a sprint. A single production outage that could have been caught by the integration test stage would cost far more than the monthly pipeline bill.

Practice	Details
Keep build times under 10 minutes	Developers will not wait 30 minutes for feedback. Use parallel test suites, Maven caching, Docker layer caching.
Never push directly to main	All changes go through a PR with at least 1 reviewer. Branch protection rules enforce this in GitHub.
Fail fast in the pipeline	Run the fastest, cheapest checks first: unit tests (30s), then build (2min), then integration (8min), then prod deploy (10min).
Every build is immutable	Each build produces a Docker image tagged with the git SHA (e.g., product-svc:abc1234). Never override a previous build.
Blue/green for stateless services	ECS services are stateless (state in Aurora/DynamoDB/Redis). Blue/green deployment with 10-minute cutover.
Feature flags for risky features	Deploy the code to 100% of production, but reveal the feature to 1% of users. Monitor. Ramp up. In production.
Scan for CVEs in the pipeline	Integrate OWASP Dependency Check and ECR image scanning. Fail the build if HIGH severity CVEs are found.

ShopFlow uses four environments. Code always flows in one direction: local -> dev -> staging -> prod. A change can never skip an environment. Each environment is a separate AWS account for billing isolation and blast radius reduction.

Environment	AWS Account	Purpose	Who Deploys	Deployment Trigger
Local	N/A (laptop)	Individual developer testing; Docker development	Developer	Manual
Dev	AWS Account 1	Integration with real AWS services; automated testing	CodePipeline (feature branches)	Push to feature branch
Staging	AWS Account 2	Production-like environment; final validation	CodePipeline (prod branch)	Push to prod branch
Production	AWS Account 3	Live customer traffic; 1M MAU; 99.9% uptime	CodePipeline (main branch)	After manual approval by SRE lead

INFO Why Separate AWS Accounts?

ShopFlow uses separate AWS accounts for dev/staging/prod (not just separate VPCs). This provides: (1) billing isolation — dev costs show separately from prod costs; (2) IAM isolation — a bug in dev CDK code cannot affect prod resources because the AWS accounts are separate; (3) service limit isolation — a dev load test does not consume prod EC2 capacity limits; (4) CloudTrail audit — prod account audit logs are clean (no dev noise).

ShopFlow's engineering team tracks pipeline status in Slack. Every pipeline event (success, failure, manual approval needed) sends a Slack message via SNS -> Lambda -> Slack webhook.

```
// CDK: Pipeline notifications via SNS + ChatBot (native Slack integration) // Option 1: AWS
Chatbot (recommended — native integration, no Lambda needed) import
software.amazon.awscdk.services.chatbot.*; // Create Slack channel configuration
SlackChannelConfiguration slackChannel = SlackChannelConfiguration.Builder.create(this,
"SlackPipeline") .slackChannelConfigurationName("shopflow-pipeline-notifications")
.slackWorkspaceId("T0XXXXXX") // Slack workspace ID .slackChannelId("C0YYYYYYY") //
#deployments Slack channel ID .notificationTopics(List.of(pipelineNotificationTopic)) .build();
// Pipeline notification rule: notify on specific pipeline events
NotificationRule.Builder.create(this, "PipelineNotification")
.notificationRuleName("ShopFlow-Pipeline-Events") .source(pipeline) // CodePipeline pipeline as
the source .events(List.of( PipelineNotificationEvents.PIPELINE_EXECUTION_FAILED,
PipelineNotificationEvents.PIPELINE_EXECUTION_SUCCEEDED,
PipelineNotificationEvents.MANUAL_APPROVAL_NEEDED,
PipelineNotificationEvents.STAGE_EXECUTION_FAILED)) .targets(List.of(slackChannel)) .build();
// Slack messages look like: // FAILED: shopflow-product-svc-pipeline > Build stage > CodeBuild
failed // -> Link to failed build log in CodeBuild console // APPROVAL NEEDED: prod deploy is
waiting for approval // -> Link to CodePipeline console to approve // SUCCEEDED:
shopflow-product-svc-pipeline completed -> product-svc:abc1234 deployed to prod
```

ECS Rolling vs Blue/Green — Choosing the Right Strategy

ECS supports two deployment strategies: Rolling Update (built-in, simpler) and Blue/Green (via CodeDeploy, safer). ShopFlow uses Blue/Green in production and Rolling in dev/staging.

Aspect	Rolling Update	Blue/Green (CodeDeploy)
How it works	Replace old tasks with new tasks one by one; new versions complete new tasks (green); old ALB traffic all at once	Creates complete new task set (green); switch ALB traffic all at once
Traffic during deploy	Both old and new versions serve traffic simultaneously during rolling update	100% old traffic rolling green causes version instantly switches to 100% new
Rollback speed	Must wait for new deployment to roll back (slow, minutes)	Instantly switch ALB weight back to blue (< 30 seconds)
Observation window	None — no way to automatically rollback after deployment	Deployment window with CloudWatch alarms; auto-rollback if alarm
Setup complexity	Built into ECS; no extra services	Requires CodeDeploy deployment group + appspec.yaml + task c
ShopFlow use	Dev and staging environments	Production (safety and auto-rollback are worth the setup complexity)
Cost	No extra cost	CodeDeploy for ECS blue/green is free

GitHub Actions vs CodePipeline — When to Choose Each

Many teams use GitHub Actions instead of CodePipeline. Understanding the tradeoffs helps you choose the right tool for each project.

Aspect	GitHub Actions	AWS CodePipeline
Execution environment	GitHub-hosted runners (Ubuntu, macOS, Windows) or self-hosted	AWS CloudBuild (managed, serverless; Linux only)
AWS integration	Good: use aws-actions/ marketplace actions; OIDC for native CodeBuild (has original AWS role; no credential ma	Native: CodeBuild (has original AWS role; no credential ma
Workflow as code	YAML in .github/workflows/ (version-controlled, branched, PR control)	CDK, BR Control (CDK version-controlled; Console is not)
Cost	Free for public repos; \$0.008/min for private repos (Linux)	\$0.005/min CodeBuild + \$1/pipeline/month CodePipeline
Multi-account deploy	Possible with OIDC + IAM roles per account	Native: IAM role chaining between accounts
ShopFlow recommendation	Use GitHub Actions for: open-source projects, developer tooling, CI/CD for production	Use CodePipeline for: production deployments, reusable AWS, ba

Canary Deployments — Gradual Traffic Shifting

A canary deployment sends a small percentage of production traffic to the new version before shifting all traffic. If the canary shows errors, traffic shifts back. CodeDeploy ECS supports three traffic-shifting configurations.

Configuration	Traffic Shifting	Observation Period	ShopFlow Use Case
EcsAllAtOnce	100% traffic to new version immediately	Full deployment time	Dev/staging: fastest; acceptable risk
EcsLinear10PercentEvery1Minute	10% more traffic added every minute until 100%	10 minutes total	Low-risk changes: config updates, minor
EcsCanary10Percent5Minutes	10% to new version for 5 min; then 100%	5 minutes at 10% (canary phase)	ShopFlow checkout changes: catch erro

```
// CDK: Configure canary deployment for product-svc
EcsDeploymentGroup deploymentGroup =
EcsDeploymentGroup.Builder.create(this, "ProductSvcDG") .service(productFargateService)
    .deploymentConfig( // CodeDeploy built-in: 10% canary for 5 min, then 100%
EcsDeploymentConfig.CANARY_10_PERCENT_5_MINUTES)
    .blueGreenDeploymentConfig(EcsBlueGreenDeploymentConfig.builder()
        .blueTargetGroup(blueTargetGroup) .greenTargetGroup(greenTargetGroup) .listener(httpsListener)
        .terminationWaitTime(Duration.minutes(10)) .build()) .alarms(List.of(albHighErrorAlarm)) //
    auto-rollback if 5xx alarm fires during canary .autoRollback(AutoRollbackConfig.builder()
        .deploymentInAlarm(true).build()) .build(); // Result: 5 minutes where 10% of users hit the new
    product-svc version // If no 5xx spike during those 5 minutes -> 100% of traffic shifts to new
    version // If 5xx spike -> ALB automatically routes back to 100% blue (old version)
```

Question	Answer
My build takes 30 minutes — how do I speed it up?	Identify the slowest phase with CodeBuild phase logs. Usually: Maven downloads (fix: S3 cache)
How do I deploy to multiple AWS accounts using AWS IAM roles from one pipeline?	The tools account assumes a role in each target account
Should database migrations run in the pipeline or application code?	ShopFlow uses Flyway in Spring Boot: migrations run at application startup
How do we prevent two pipeline runs from updating production?	CodePipeline Execution Mode? "Superseded" (latest push wins; in-progress run is cancelled), "

CodeBuild can parse JUnit XML test output and display test results directly in the CodeBuild console — pass/fail per test, duration, and trend over time. ShopFlow uses this to track test health across builds.

```
# buildspec.yml: configure test reporting phases: build: commands: # Run tests with XML output
for CodeBuild reports - mvn test -Dsuresfire.useFile=true # Maven Surefire writes to
target/surefire-reports/ reports: ShopFlowTestReport: files: -
"target/surefire-reports/TEST-*.xml" file-format: JUNITXML discard-path: yes # In CodeBuild
console: # Build -> Test tab shows: # - Total tests: 412 # - Passed: 409 | Failed: 3 | Skipped:
0 # - Failed tests listed by name with stack trace # - Duration trend graph (is test suite
getting slower over time?) # - Test history per test case (flaky tests visible as intermittent
failures)
```

Test Category	Framework	Count	Avg Duration	Runs In
Unit Tests — Service Layer	JUnit 5 + Mockito	280 tests	< 1ms each (mocked)	Build stage (pre_build)
Unit Tests — Repository Layer	JUnit 5 + H2 in-memory	65 tests	< 5ms each	Build stage (pre_build)
Integration Tests — REST APIs	Spring Boot Test + Rest Assured	45 tests	50-500ms (real HTTP)	Integration Test stage
Integration Tests — DynamoDB	JUnit 5 + DynamoDB Local	22 tests	10-100ms	Integration Test stage
Smoke Tests — Staging	Shell script + curl	10 endpoints	1-3s each	Staging Deploy stage

The CodePipeline and CodeBuild execution roles need carefully scoped IAM permissions. The pipeline should only be able to deploy to the services it manages — not to modify IAM roles, delete databases, or access other teams' resources.

```
// CDK: Least-privilege IAM role for CodeBuild (build stage) Role codeBuildRole =
Role.Builder.create(this, "CodeBuildRole").assumedBy(new
ServicePrincipal("codebuild.amazonaws.com")).build(); // Only allow push to the specific ECR
repos the pipeline manages codeBuildRole.addToPolicy(PolicyStatement.Builder.create()
.actions(List.of("ecr:GetAuthorizationToken", "ecr:BatchCheckLayerAvailability",
"ecr:PutImage", "ecr:InitiateLayerUpload", "ecr:UploadLayerPart", "ecr:CompleteLayerUpload"))
.resources(List.of( productSvcRepo.getRepositoryArn(), // only product-svc ECR repo
orderSvcRepo.getRepositoryArn())) // only order-svc ECR repo .build()); // Only allow SSM
Parameter Store read for build-time secrets
codeBuildRole.addToPolicy(PolicyStatement.Builder.create().actions(List.of("ssm:GetParameter",
"ssm:GetParameters")).resources(List.of( // Only the specific parameters needed for build; not
all SSM parameters "arn:aws:ssm:us-east-1:123456789:parameter/shopflow/cicd/*")) .build()); //
NOT granted: iam:*, ec2:*, rds:*, cloudformation:DeleteStack // The build stage cannot delete
infrastructure or escalate IAM privileges
```

CodePipeline and CodeBuild publish metrics to CloudWatch. ShopFlow monitors these to detect stuck pipelines, build time regressions, and deployment frequency trends.

Metric	Namespace	What to Alarm On	ShopFlow Threshold
PipelineExecutionDuration	AWS/CodePipeline	Duration increasing over time (builds getting slower)	Alert if > 45 minutes (normal: 25-35 min)
FailedPipelines	AWS/CodePipeline	More than N failures in an hour (broken build quality)	Alert if > 3 failures/hour
BuildDuration	AWS/CodeBuild	Build time creeping up (dependency or test bloat)	Alert if > 12 minutes (normal: 6 min)
SucceededBuilds + FailedBuilds	AWS/CodeBuild	Track pass rate trend over time	Alert if pass rate drops below 80% over 7-days
DeploymentFrequency (ShopFlow)	ShopFlow/Pipeline	Track how often team is deploying (DORA metric)	Goal: > 10 deployments/day; alert if < 5 over 7-days

16-27

DORA Metrics — Measuring CI/CD Performance

DORA (DevOps Research and Assessment) defines four key metrics for measuring software delivery performance. ShopFlow tracks all four using custom CloudWatch metrics and Logs Insights queries.

DORA Metric	Definition	ShopFlow Current	Elite Performer Benchmark
Deployment Frequency	How often the team deploys to production	15-20 times/day	On-demand (multiple per day)
Lead Time for Changes	Time from code commit to production	28-42 minutes (pipeline duration)	< 1 hour
Change Failure Rate	% of deployments that cause a production incident	2.3% (4-5 auto-rollback per month)	< 5%
Mean Time to Restore (MTTR)	Time to recover from a production failure	8 minutes (auto-rollback) or 35-min (manual revert + redeploy)	< 1 hour

SUCCESS Elite Performer Status

ShopFlow meets the DORA "Elite Performer" benchmark for Deployment Frequency (multiple per day) and Lead Time for Changes (< 1 hour). The Change Failure Rate at 2.3% is within the "High Performer" range. MTTR of 8 minutes for auto-rollback qualifies as Elite. These metrics directly translate to business value: faster features, faster bug fixes, and less downtime for ShopFlow customers.

16-28

Chapter Summary — ShopFlow CI/CD Pipeline

You now understand how ShopFlow deploys 15-20 times per day with confidence. Here is a summary of the key tools and concepts.

Concept	Key Takeaway
CI/CD purpose	Automate build, test, and deploy so engineers focus on features, not deployment mechanics. Small, frequent releases.
CodeBuild buildspec.yml	Four phases: install (runtime setup), pre_build (unit tests + ECR login), build (Docker build + push), post_build (optional).
CodeDeploy blue/green for ECS	New version (green) starts alongside old (blue). ALB switches traffic instantly. 10-minute observation window.
Manual approval gate	SRE lead approves prod deployment after verifying staging. CodePipeline sends SNS email. Without approval, deployment fails.
Rollback options	Auto-rollback (< 2 min, during observation window), pipeline stop (bad code never reaches prod), git revert + new build.
Pipeline security	CodeBuild IAM role has least privilege: only ECR push for specific repos, only SSM parameter read for pip install.
DORA metrics	Track Deployment Frequency, Lead Time, Change Failure Rate, MTTR. ShopFlow meets Elite Performer benchmark.

16-29

AWS CodeArtifact — Private Maven Repository

CodeArtifact is a fully managed artifact repository (like a private Maven Central) that stores internal shared libraries. ShopFlow uses it to share the shopflow-commons library (domain objects, shared utilities) across all microservices without publishing to Maven Central.

```
// CDK: Create CodeArtifact domain and repository
CfnDomain domain =
CfnDomain.Builder.create(this, "ArtifactDomain").domainName("shopflow").build();
CfnRepository internalRepo = CfnRepository.Builder.create(this, "InternalRepo")
```

```
.domainName(domain.getDomainName()) .repositoryName("shopflow-internal") .description("ShopFlow
internal Maven artifacts: shopflow-commons, shopflow-api-models")
.externalConnections(List.of("public:central")) // proxy Maven Central .build(); //
buildspec.yml: configure Maven to use CodeArtifact phases: install: commands: # Get
CodeArtifact auth token (valid 12 hours) - export CODEARTIFACT_TOKEN=$(aws codeartifact
get-authorization-token --domain shopflow --domain-owner 123456789 --query authorizationToken
--output text) # Set up Maven settings.xml to use CodeArtifact - mvn --settings ci-settings.xml
... # settings.xml uses $CODEARTIFACT_TOKEN // shopflow-commons pom.xml: publish to
CodeArtifact // distributionManagement section points to CodeArtifact repository URL // mvn
deploy publishes to CodeArtifact; other services pull from it // Benefits: // - Internal
libraries not exposed on Maven Central (proprietary code) // - Dependency proxying:
CodeArtifact caches Maven Central; builds work even if internet is down // - Consistent artifact
versioning across all microservice builds
```

16-30

Full Pipeline Walkthrough — commit to production

Let's walk through a complete ShopFlow deployment from a developer's perspective — from writing code to it running in production.

■ Full ShopFlow Deployment Walkthrough
1. Engineer creates feature branch: git checkout -b feature/add-product-rating
2. Writes code + unit tests in IntelliJ IDEA; runs locally: mvn test (all 412 tests pass)
3. Pushes branch: git push origin feature/add-product-rating
4. Opens PR on GitHub: title "Add product rating to search results"
5. GitHub automatically triggers CodePipeline on the feature branch: unit tests run in 4 minutes
6. Green CI check appears on PR -> Teammate reviews and approves the PR
7. Engineer merges PR to main: git merge (GitHub squash merge)
8. CodePipeline detects push to main -> Pipeline starts (Stage 1: Source)
9. Stage 2 Build: CodeBuild builds JAR + Docker image + pushes to ECR in 6 minutes
10. Stage 3 Integration Test: REST Assured tests run against real DynamoDB/Aurora in 10 minutes
11. Stage 4 Staging Deploy: Blue/green to staging; smoke tests pass; Slack: "Staging deploy succeeded"
12. Stage 5: SRE lead gets email + Slack: "Approve prod deploy? Check staging.shopflow.com/products/PROD-1"
13. SRE verifies rating shows on staging; clicks Approve in CodePipeline console
14. Stage 5 Prod Deploy: Blue/green to prod; 10% canary for 5 minutes; no 5xx spike
15. Full 100% traffic shift; Slack: "#deployments product-svc:abc1234 deployed to prod"
16. Total time: commit at 2:00 PM -> production at 2:38 PM (38 minutes)

16-31

Black Friday Pipeline Freeze — Change Freeze Policy

On Black Friday, ShopFlow enforces a change freeze: no non-critical deployments to production. This reduces the risk of introducing bugs during peak traffic. The CI/CD pipeline is modified to enforce this.

Aspect	Normal Policy	Black Friday Policy (Thu 6PM - Mon 9AM)
Prod deployments	Any approved change deploys	Only P0 (critical bug fixes) deployments allowed; requires 2 SRE approval
Manual approval	1 SRE lead approval	2 SRE approvals required; on-call SRE + VP Engineering
Pipeline override	N/A	Emergency deploy bypasses change freeze only with VP approval and

Aspect	Normal Policy	Black Friday Policy (Thu 6PM - Mon 9AM)
Staging deployments	Normal	Allowed (staging is not customer-facing)
Feature flag changes	Via normal deploy	Allowed via AppConfig feature flag update (instant, no deploy needed)
Database changes	Pipeline + Flyway	BLOCKED: no Aurora schema migrations during Black Friday

WARN Change Freeze Enforcement

ShopFlow enforces the change freeze in GitHub branch protection rules: the main branch requires an additional approval from a "change-control" GitHub team during the freeze window. This is configured via a GitHub Actions workflow that checks the current date/time and applies additional required reviewers. The CDK pipeline also has an additional manual approval stage (VP Engineering) that is added 48 hours before Black Friday via a `cdk deploy -c blackFriday=true` context flag.

16-32

Pipeline Quick Reference — Commands and Artifacts

A concise reference for the commands, files, and artifacts used throughout the ShopFlow CI/CD pipeline.

Item	Value / Description
Trigger for pipeline	Push to main branch in GitHub -> CodePipeline Source stage starts automatically
Build environment	Amazon Linux 2023, Java 21 (Amazon Corretto), Docker 24, Maven 3.9
Docker image tag format	ECR_URI/product-svc:abc1234 (7-char git SHA) + :latest
imagedefinitions.json	[{"name":"app","imageUri":"123.ecr.../product-svc:abc1234"}] — tells CodeDeploy which images to use
appspec.yaml	Instructs CodeDeploy: task definition + container name + port + lifecycle hooks
taskdef.json	ECS task definition template; contains IMAGE1_NAME placeholder
Blue/Green observation window	10 minutes (configurable); auto-rollback if CloudWatch alarm fires
Rollback command (manual)	git revert HEAD && git push origin main — triggers new pipeline run
Pipeline duration	~35 minutes: Source(1) + Build(6) + IntTest(10) + StagingDeploy(8) + Approval(varies) + ProductionDeploy(10)
CodeBuild cache location	S3 bucket: shopflow-codebuild-cache/maven-cache
Approval notification	SNS -> email to sre-lead@shopflow.com + Slack #deployments channel
DORA Deployment Frequency	15-20/day (Elite Performer)
DORA Lead Time	28-42 min commit -> production (Elite Performer: < 1 hour)

16-33

Continuous Compliance — AWS Config and Pipeline Gates

ShopFlow must comply with PCI-DSS (payment card data) and SOC 2. Compliance checks are embedded directly into the CI/CD pipeline so non-compliant infrastructure never reaches production.

Compliance Check	Tool	Pipeline Stage	What It Enforces
Infrastructure policy validation	Open source CDK Nag	Build stage	Detects non-compliant CDK patterns: public S3 buckets, unencrypted RDS instances
AWS Config Rules	AWS Config + conformant	Post-deploy (continuous)	PCI-DSS conformance pack checks: encryption at rest, no public access
License compliance	FOSSA (or Black Duck)	Build stage	Ensures all open-source dependencies have compatible licenses (MIT, Apache, etc.)
Secret rotation	AWS Secrets Manager + CloudFormation	Continuous (separate pipeline)	Automatically rotates every 30 days automatically. If manual rotation is needed, it can be triggered via CLI or API.

```
// Add cdk-nag to CDK app for continuous compliance scanning // pom.xml: add
software.amazon.cdknag:cdk-nag dependency import io.github.cdklabs.cdknag.*; // Apply PCI-DSS
checks to entire app Aspects.of(app).add(new AwsSolutionsChecks(NagPackProps.builder()
.verbose(true) // show rule IDs in output .build())); // This will fail cdk synth if: // - Any
S3 bucket allows public access -> AwsSolutions-S2 // - Any RDS instance is not encrypted ->
```

```
AwsSolutions-RDS3 // - Any IAM policy uses wildcard actions -> AwsSolutions-IAM4 // - Any
Security Group allows 0.0.0.0/0 on non-80/443 ports -> AwsSolutions-EC23 // - ECS task role has
AdministratorAccess -> AwsSolutions-ECS7 // Suppress a specific rule if intentional (with
justification required) NagSuppressions.addResourceSuppressions(publicAssetsBucket,
List.of(NagPackSuppression.builder() .id("AwsSolutions-S2") .reason("ShopFlow static assets
(product images) are intentionally public read-only") .build())); // Pipeline: cdk synth will
now fail if any cdk-nag rule fires // Compliance is enforced at infrastructure-as-code time, not
after deployment
```
